SALUNIC - Sistema de Salud y Bienestar

App Tkinter + Módulo por Consola Integrados

DataClass | Cola con Prioridad | Búsqueda Lineal | Archivo JSON

Guía para Estudio, Exposición y Defensa

Elaborado para comprensión y apropiación del código

ÍNDICE

1. Objetivo de esta guía
2. Estructura actual del proyecto
3. ¿Cómo se conectan App, Consola y core/?
4. DataClass (entidades.py)
5. Cola con Prioridad con heapq (cola_atencion.py)
6. Búsqueda Lineal (busqueda_lineal.py)
7. Persistencia con Archivo JSON (archivo_datos.py)
8. Integración en la App Tkinter
9. Panel de Atención (Integración visual de la Cola)
10. Flujo completo: Agregar a cola -> Ver -> Atender
11. Tips para exponer y defender el código
12. Checklist antes de exponer
13. Conclusión

1. Objetivo de esta guía

Esta guía tiene como finalidad entender paso a paso cómo está estructurado el proyecto SALUNIC ahora, cómo se unificó el módulo por consola con la aplicación gráfica Tkinter mediante la carpeta core/, y cómo explicar cada parte con seguridad al momento de exponer o defenderlo.

2. Estructura actual del proyecto

Raiz del proyecto:

- app/ -> Interfaz grafica Tkinter (Login, Registro, Citas, Home, Panel de Atencion)
- consola/ -> Modulo por consola (menu, pruebas de logica)
- core/ -> LOGICA COMPARTIDA (DataClass, Cola con Prioridad, Busqueda Lineal, Archivo JSON)
- docs/ -> Documentacion, diagramas, capturas, PDFs
- scripts/ -> Scripts auxiliares
- run_app.bat / run_consola.bat / README.md

Clave:

core/ es el corazón del proyecto. Tanto app/ como consola/ importan desde ahí. Así evitamos duplicar código y demostramos reutilización.

3. ¿Cómo se conectan App, Consola y core/?

La idea es sencilla: toda la lógica que pide el profesor vive en core/.

- consola/ importa desde core/ (entidades, datos, funciones, cola_atencion, busqueda_lineal, archivo_datos)
- app/ importa desde core/ (login usa busqueda_lineal, registro usa entidades.Usuario, citas/atención usan cola_atencion y archivo_datos)

Gracias a esto, ambos módulos trabajan con las MISMAS clases, MISMAS listas y MISMO archivo salunic_datos.json.

¿Qué demuestra esto?

Que entendimos el objetivo: integrar la lógica al sistema, no tener dos programas aislados. La app gráfica ahora hace uso real de lo implementado en el módulo por consola.

4. DataClass (entidades.py)

Ubicación: core/entidades.py

Usamos @dataclass para representar entidades del sistema con menos código y mayor claridad.

Clases:
- Usuario: id, nombre, email, password, telefono
- CitaMedica: id, paciente_id, medico_id, especialidad, fecha, hora, lugar
- Medicamento: id, nombre, dosis, frecuencia, presentacion
- AlmacenMedicamento: id, medicamento_id, cantidad, fecha_vencimiento

Uso en app: Al registrar, creamos entidades.Usuario(...) y lo agregamos a core.datos.usuarios para mantener la misma estructura.

5. Cola con Prioridad con heapq (cola_atencion.py)

Ubicación: core/cola_atencion.py

Importamos heapq. La clave es usar una tupla (prioridad, turno_global, item).

Reglas:
- es_adulto_mayor = True -> prioridad = 0 (MAYOR PRIORIDAD)
- es_adulto_mayor = False -> prioridad = 1 (NORMAL)
- turno_global sirve como desempate
- heapq.heappop() siempre extrae el elemento con menor valor de prioridad

Métodos: encolar(), desencolar(), mostrar_cola(), tamanio(), esta_vacia()

Uso en app: Botón "AGREGAR A COLA DE ATENCION (PRIORIDAD)" en Citas y pantalla "Panel de Atención" para atender.

6. Búsqueda Lineal (busqueda_lineal.py)

Ubicación: core/busqueda_lineal.py

Consiste en recorrer la lista elemento por elemento con un ciclo for, comparando hasta encontrar coincidencias.

Funciones:
- busqueda_lineal(lista, criterio, atributo): Genérica, usa getattr(), compara case-insensitive para strings
- busqueda_lineal_por_email(usuarios, email): Búsqueda específica por email
- busqueda_lineal_citas_por_medico(citas, medico_id): Filtra por médico
- busqueda_lineal_medicamento_por_nombre(medicamentos, nombre): Filtra por nombre

Uso en app: Login usa busqueda_lineal_por_email() para validar credenciales. También se usa para obtener datos del usuario logueado.

7. P

Ubicación: core/archivo_datos.py | Archivo: consola/salunic_datos.json (compartido)

Guardar:
- Usamos dataclasses.asdict() para convertir objetos DataClass a diccionarios
- json.dump() con ensure_ascii=False, indent=4

Cargar:
- json.load() lee el archivo
- Limpiamos listas con clear()
- Reconstruimos con Usuario(**u), CitaMedica(**c), etc.

Uso en app: Al registrar usuario y al agregar cita se guarda. Al iniciar app se intenta cargar datos.

8. Integración en la App Tkinter

Ubicación: app/

Puntos clave de integración:

- app/main.py: Al iniciar carga datos desde core.archivo_datos. Navegación entre pantallas (Splash, Login, Home, Citas, Atencion, etc.)
- app/screens/login_screen.py: Usa core.busqueda_lineal + core.archivo_datos para login
- app/screens/registro_screen.py: Crea core.entidades.Usuario, agrega a core.datos.usuarios, guarda con core.archivo_datos
- app/screens/citas_screen.py: Muestra citas desde core.datos.citas, permite AGREGAR NUEVA CITA (guarda con DataClass), agrega a cola con prioridad
- app/data/users_data.py: Adaptado para usar core como fuente principal, mantiene compatibilidad

Todo con el mismo estilo visual (gradiente, colores, tarjetas redondeadas).

9. P

Ubicación: app/screens/atencion_screen.py

Esta pantalla hace visible lo que antes solo salía en consola.

Elementos:
- EN ATENCIÓN: Muestra el paciente que se acaba de atender con su prioridad
- COLA DE ESPERA: Lista ordenada por prioridad/turno (Adulto Mayor resaltado)
- ATENDER SIGUIENTE: Llama a cola_atencion.desencolar()
- ACTUALIZAR PANEL: Refresca la vista

Se accede desde Home o directamente. Sigue el mismo formato visual del resto de la app.

10. Flujo completo: Agregar a cola -> Ver -> Atender

1. Ir a Citas -> "AGREGAR A COLA DE ATENCION (PRIORIDAD)"
2. Seleccionar si es Adulto Mayor (Sí/No). Esto define prioridad 0 o 1.
3. Se llama a core.cola_atencion.encolar(item, es_adulto_mayor)
4. Ir a Home -> "PANEL DE ATENCION CON PRIORIDAD" o pantalla Atencion
5. Ver cola ordenada (Adulto Mayor primero)
6. Presionar "ATENDER SIGUIENTE" -> desencolar() y pasa a "EN ATENCIÓN"
7. En terminal también se ven los prints [COLA] confirmando el funcionamiento

11. Tips para exponer y defender el código

- Estructura: Di primero 'core/ es la lógica compartida, app y consola la reutilizan'
- DataClass: Menciona que con @dataclass representamos entidades (Usuario, CitaMedica...)
- Cola con prioridad: Explica tupla (prioridad, turno_global, item). Adulto Mayor = prioridad 0
- Búsqueda lineal: Recorre elemento por elemento. Usado en login con busqueda_lineal_por_email
- Archivo JSON: Guardar con asdict(), cargar reconstruyendo con Clase(**dict)
- Integración: Muestra login/registro/citas usando core. No son módulos separados
- Panel de Atención: Lo más visual. Agrega, ve la cola, atiende y demuestra prioridad
- Unificación: Un solo salunic_datos.json para app y consola
- Sé breve: Explica 'qué hace' y 'por qué' con el código abierto

12. Checklist antes de exponer

[ ] Mostrar estructura de carpetas (core/, app/, consola/)
[ ] Explicar core/ como lógica compartida
[ ] Mostrar entidades.py con DataClass
[ ] Explicar cola_atencion.py con heapq y prioridad al adulto mayor
[ ] Mostrar busqueda_lineal.py usado en login
[ ] Explicar archivo_datos.py (guardar/cargar JSON)
[ ] Demo en app: Registrar/Login
[ ] Demo en app: Agregar cita
[ ] Demo en app: Agregar a cola (adulto mayor vs normal)
[ ] Demo en app: Panel de Atención -> Atender siguiente
[ ] Mostrar que imprime en terminal [COLA]
[ ] Mencionar reutilización entre app y consola

13. Conclusión

El proyecto SALUNIC ahora está integrado correctamente. La lógica solicitada (DataClass, Cola con Prioridad, Búsqueda Lineal, Archivo de Datos) se encuentra centralizada en core/ y es utilizada tanto por el módulo por consola como por la aplicación gráfica Tkinter.

El Panel de Atención permite visualizar y aplicar la cola con prioridad de forma práctica. Con esta estructura unificada, el código es más limpio, profesional, fácil de entender y está listo para exponer y defender con seguridad.